

Take-Home Assignment

The task

Build a Mini Helpdesk web app. An end-customer (no login required) submits a support ticket. Internal agents (login required) triage, respond, and close tickets. Think Zendesk or Freshdesk at the tiniest scale.

Required stack

- Frontend: React. TypeScript strongly preferred.
- Backend: Node.js + Express (or Next.js API routes). TypeScript strongly preferred.
- Database: MongoDB.
- Auth: JWT-based for agents. Seed at least two agents (e.g., `agent1@xriseai.com` and `admin@xriseai.com`) to test role-based behaviour.
- Deploy to any free-tier host.

Must-have features

1. Public ticket-submission form that captures name, email, subject, body, and priority, and returns a ticket ID.
2. Public status-check page — given a ticket ID and email, show the status and the latest agent reply. No login.
3. Agent login and agent dashboard — a paginated list of tickets with filters (at minimum: status, priority, assignee) and server-side search on subject / body.
4. Ticket detail view — timeline of events (created, reassigned, replied, status changed) with forms to add a reply and change status.
5. Role split — an agent can reply and change status. An admin can do everything an agent can, plus reassign tickets. Admins see all tickets; agents see only tickets assigned to them.

Quality bar

- A state-management approach suited to the app.
- At least one reusable, documented component.
- Loading, empty, and error states on every data fetch.
- Form validation with useful feedback. Keyboard accessible.
- Responsive layout from phone to desktop.
- Optimistic UI where it improves perceived responsiveness.
- Passwords and JWT handled safely. Auth enforced on every protected route and query.
- Server-side input validation. Errors return a consistent shape and do not leak stack traces.
- Structured logs suitable for production debugging.
- Schema and indexes aligned to the queries the app actually runs.
- Rate limiting on public endpoints.
- `.env.example` committed. Real `.env` git-ignored. CORS locked down. No secrets in the repo.
- Automated tests on the logic that matters most. *Page 1 of 2*

Architecture document (ARCHITECTURE.md)

Include a 2-page ARCHITECTURE.md in the repo root covering:

- A system diagram. Show frontend, backend, DB, and anything external you'd add next.
- Data model, indexes, and one schema tradeoff made and why.
- Auth and authorization model — how a request is proved legitimate, and how cross-agent data access is prevented.
- Scaling — one paragraph each on what breaks first at 1,000 tenants / 1M tickets, what breaks first at 100 concurrent agents, and what you'd change.
- Observability — what to log, measure, and alert on in production.
- Top 3 items you'd build in week 2–3.

Have

- Dockerfile + docker-compose.yml.
- Real-time ticket updates.
- File attachments on tickets.
- CI pipeline that lints and tests on every push.

AI bonus

Add one AI-powered feature to the agent dashboard. Examples:

- Draft a reply to the customer from the ticket history.
- Summarise a ticket with a suggested priority.
- Auto-classify incoming tickets into categories you design.

API key options

Free tiers that do not require a paid plan:

- **Google Gemini API** — ai.google.dev
- **Groq Cloud** — console.groq.com
- **OpenRouter** — openrouter.ai
- **Ollama (local)** — ollama.com

Document your provider choice in the README. Do not commit API keys.

Submission

6. Reply to the email sent from career@xriseai.com. Include your full name in the subject line.
7. Public GitHub repo link.
8. Live URL on any free-tier host. If you cannot deploy a live URL, mention it in your reply and be prepared to demo locally during the interview.
9. README covering: how to run locally, env vars (ship a `.env.example`), architecture decisions, and known bugs.
10. Optional: a 2–3 minute Loom walkthrough.

Timeline

- Submission deadline: Friday, 24 Aug 2026

